SCHISM on NVIDIA Grace

NCHC Open Hackathon 2026 · Final Presentation · 2026-11-04

Taiwan's operational ocean forecast: 180 s → 84 s on the same 4 Grace nodes

Baytech Environment & Engineering Inc. | 西灣海環科技股份有限公司

OpenACC
More Science, Less Programming

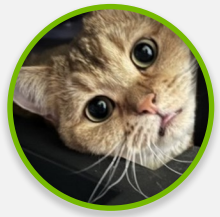
Team Baytech

Baytech Environment & Engineering Inc. · Kaohsiung, Taiwan · NSYSU marine-engineering spin-off

Team Members



Chihung Wang
CTO · Team Lead
Baytech



Chiaying Tung
Project Manager
Baytech



Haoyu Cheng
Project Engineer
Baytech



Yihsuan Kuan
Project Engineer
Baytech

Team Mentors

photo

Shijie Wang
NVIDIA

We build and run Taiwan's operational ocean current forecast (OCM) for the Central Weather Administration — every day.



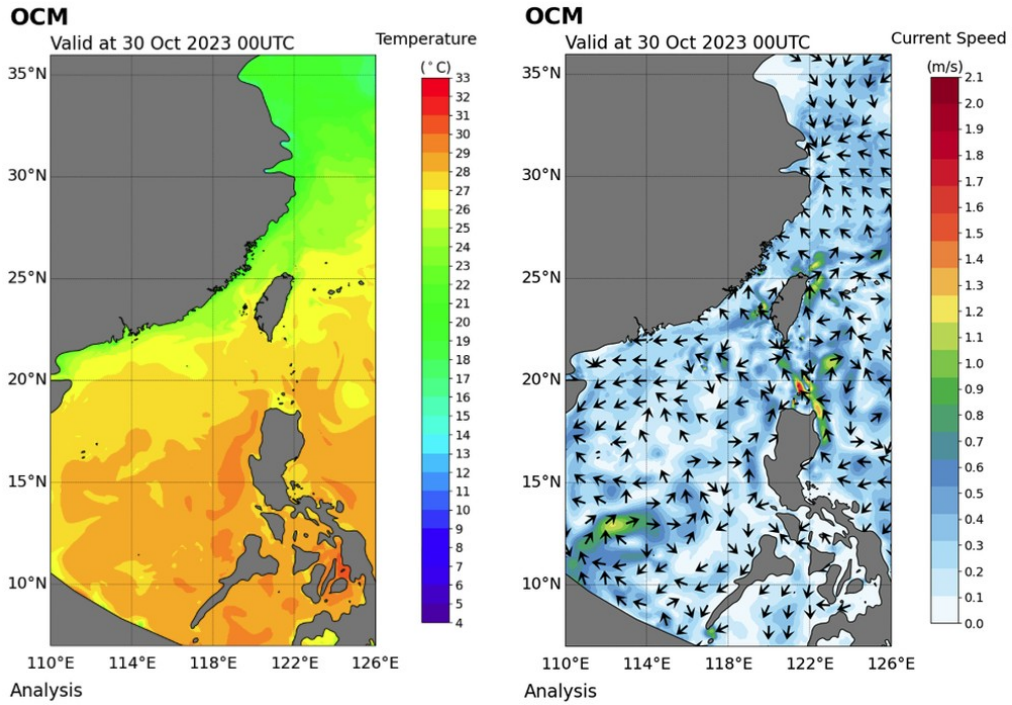
NSYSU Dept. of Marine
Environment & Engineering

BayTech environment & engineering
西灣海環科技股份有限公司

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

SCHISM — The Model Behind Taiwan's Daily Ocean Forecast

Open-source 3D unstructured-grid ocean model · Baytech built and runs the CWA Ocean Current Model on it



OCM daily analysis — sea-surface temperature and currents, Western Pacific domain

508k nodes · 48 levels

8-day forecast every day · 1.6 h on 4 nodes · fixed delivery window

Solver	implicit water level, PCG **MPI latency**
Backtracking	70 M Lagrangian trajectories **pure compute**
Transport	TVD tracer advection, 20 sub-steps **memory bandwidth**

Zhang et al. (2016), Ocean Modelling · CWA OCM: coastal 200–500 m, open ocean 8 km, forced by CWA-WRF and TGFS

OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA

Day 0 Goals → Final Day

Primary: make the production case faster on Grace CPU · Stretch: open a GPU path

1 Baseline on Grace CPU

DONE

Grace 2.0× → 1.1× faster than Xeon 8480+, 1–10 nodes

2 Profile & tune on CPU

DONE

8 optimizations · 180 → 84 s · 2.15× · CWA accepted

3 CPU → GPU trial

2 SECTIONS PORTED

Transport + Backtracking on one B200 ≈ 340 Grace cores

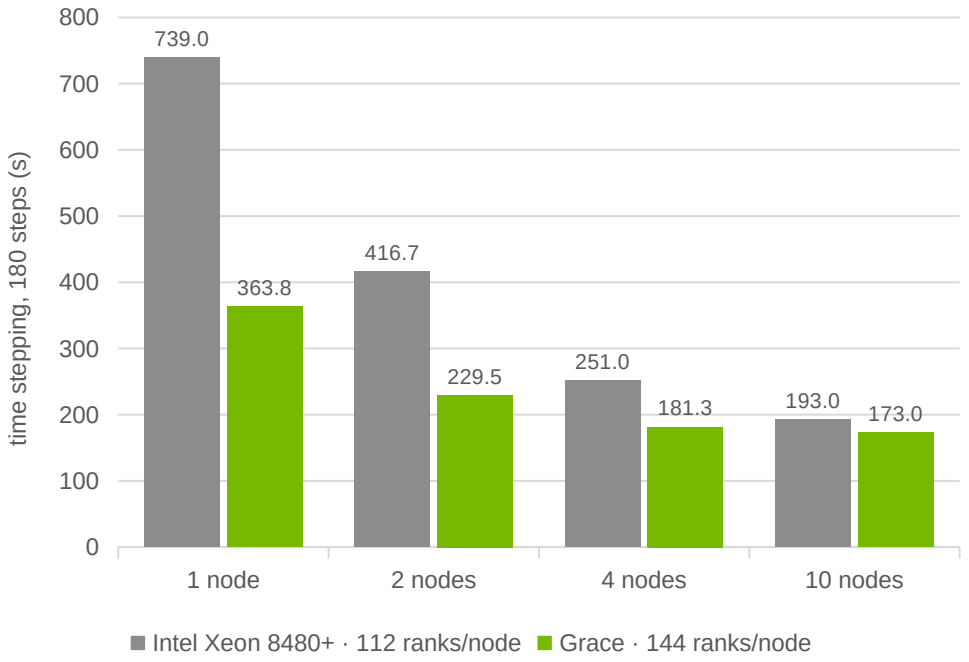
What changed on the way

- **Forcing** — parked on Day 1 as “I/O, out of scope” — became the single biggest win (−87%)
- **Solver** — from tuning the collective to changing the algorithm: block IC(0) + single-reduction CG
- **GPU** — GB200 access came in week 3: two sections ported in CUDA C++ behind a Fortran bind(C) interface, not OpenACC

One rule all three weeks: same-node A/B, one compile switch per change, bit-identical first.

Grace vs Xeon 8480+ — Same Case, Untuned, All Cores Busy

Baseline for the whole project: 4 Grace nodes × 144 ranks, 181 s per 180 steps



2.03× → 1.12×

Grace faster than Xeon, 1 → 10 nodes

- **Compute 2.1× faster** — memory bandwidth 874 vs 396 GB/s
- **Gap closes in the solver** — cross-node allreduce ~2.5× slower
- **Starting point:** 181 s per 180 steps on 4 Grace nodes

time stepping, 180 steps · Intel 112 / Grace 144 ranks per node

Xeon 8480+ 2×56 cores, oneAPI 2024 + Intel MPI · Grace 2×72 Neoverse V2, nvhpc 25.5 + HPC-X · same executable settings, -O2

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Where the Time Went — and What Each Section Got

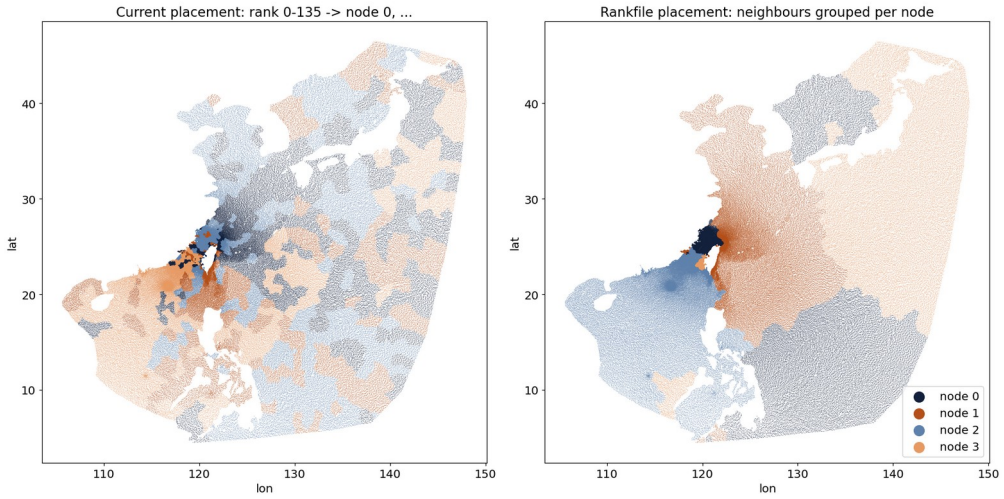
One time step, five sections · different bottleneck → different fix · same 4 nodes, 180 steps

Forcing	Backtracking	Solver	Transport	Other
I/O	compute	MPI latency	bandwidth + sync	—
37 → 4 s	**40 → 32 s**	**44 → 5 s**	**43 → 30 s**	**15 → 13 s**
−89%	**−20%**	**−89%**	**−32%**	**−13%**
record cache	reuse z-levels	placement + new CG	no memcpy, masks	not touched

180 s → 84 s on the same four nodes — 2.15× — eight changes, each behind its own compile switch.

Rank Placement — −22.7% Without Touching the Code

Opt 1 + 2: idle cores absorb OS noise, a topology-aware rankfile keeps neighbouring sub-domains on one node



Left: default mapping · Right: each sea on one node (spectral bisection → mpirun --rankfile)

−22.7%

zero lines of Fortran changed

- **136 ranks per node instead of 144** — OS noise leaves the critical path: −13%
- **Topology-aware rankfile** — cross-node halo 45% → 7%: −10%
- **Solver time 45 → 18 s** — all of it waiting, none of it compute

Placement alone is 40% of everything we saved — before a single line of code was touched.

Same nodes, two runs each · allreduce max 69 → 32 ms · CG iteration 152 → 84 µs

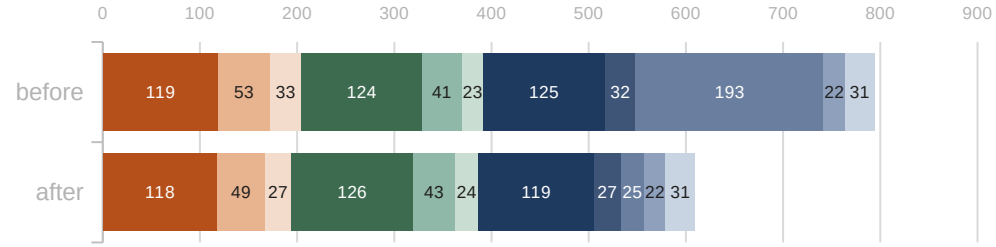
OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA

Forcing Cache — The Biggest Single Win Was in the I/O We Had Parked

Opt 6: SCHISM re-read the same hourly meteorology record on every 120 s step — 30 times per hour

−87%

Forcing time · time step −21.7% · bit-identical



nsys, rank 41, ms per step: Forcing 194 → 25, every other section unchanged

- **Cause** — rank 0 opens, reads and broadcasts 1 MB, 14 times per step
- **Fix** — a 32-record LRU cache after the broadcast (-DSFLUX_CACHE)
- **Lesson** — profile every section every week; “out of scope” hid 27 s

A/B same nodes, 3 runs each	before	after
Forcing (s)	30.0	**3.9 (−87%)**
Time step (s)	124.4	**97.4 (−21.7%)**
Station files / 3D T, S	byte-identical	byte-identical

27 of the 96 seconds we saved sat in a routine we had marked “out of scope”.

A/B same nodes, 3 runs each · CHECK build re-reads every cache hit: 1,951 hits, 0 mismatches

OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA

Solver — Sync-Count Bound, So We Changed the Algorithm

One CG iteration: 11 µs compute, 99 µs waiting · 1,084 iterations per step · Opt 7 + 8 attack both factors

1,084 → 339

iterations per step

Opt 7 · block IC(0) preconditioner, own rows only — no new communication

3 → 1

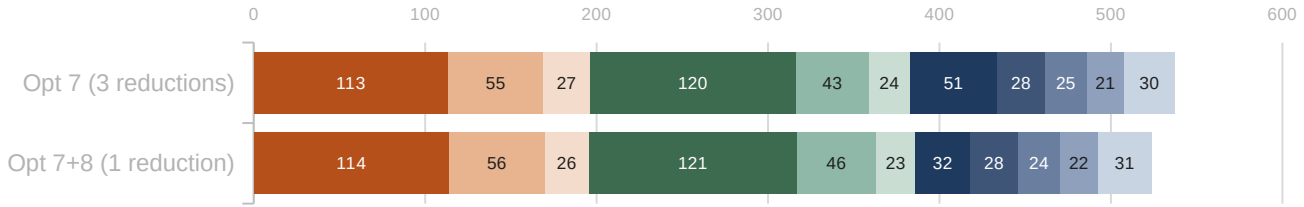
allreduce per iteration

Opt 8 · single-reduction CG — the three dot products travel in one message

117 → 32 ms

solver per step

44 s → 5 s per 180 steps · inside the model's own partition noise · CWA 96 h passed



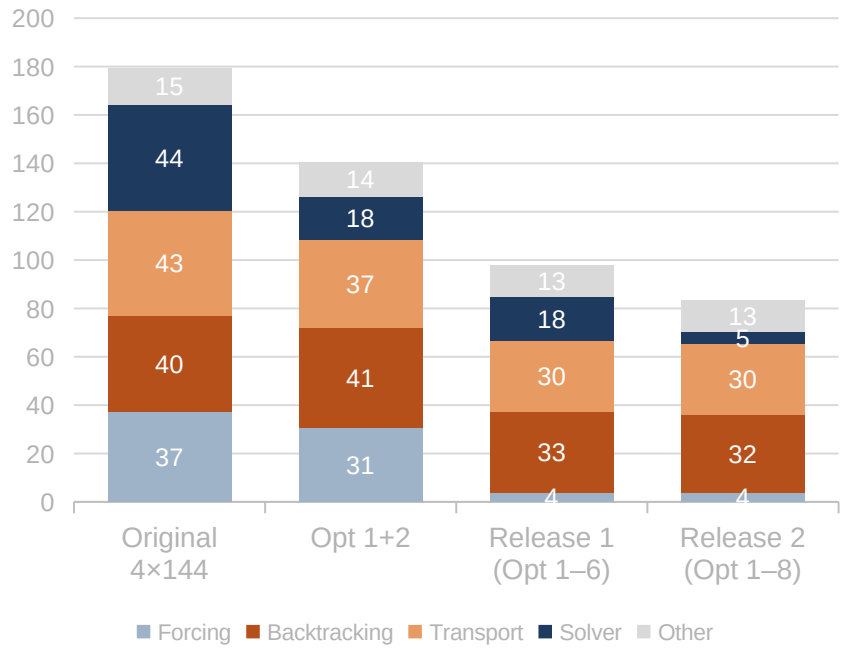
nsys, rank 41, ms per step — Opt 7 vs Opt 7+8: solver 51 → 32, everything else unchanged

Jacobi-PCG, 508k unknowns · offline preconditioner study: none 36,487 · Jacobi 1,107 · block IC(0) 343 · exact LU 183 · elevation RMSE vs reference 1.18 mm (limit 10 mm)

OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA

Result — 180 s → 84 s on the Same Four Nodes

Placement and the solver cut communication, the code changes cut compute · two releases packaged



seconds per 180 steps · Original → placement → Release 1 (Opt 1–6) → Release 2 (Opt 1–8)

2.15×

time stepping · 180.1 → 83.6 s

1.60 h → 0.74 h

8-day operational forecast on 4 nodes

CWA 96 h · Passed

1.18 mm elevation RMSE (original 1.38 mm, limit 10 mm)

Release 1 · Opt 1–6 · 1.86× · bit-identical Release 2 · Opt 1–8 · 2.15× · fastest — inputs unchanged

OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA

The Two Hackathon Metrics

Workload speedup by CPU and GPU · Workflow speedup by AI agent

Workload speedup · NVIDIA Grace CPU

2.15×

- same 4 nodes, production case, CWA accepted
- 8-day forecast 1.60 h → 0.74 h
- Grace vs Xeon 8480+: 2.0× → 1.1× untuned
- **GPU:** 1 B200 ≈ 340 Grace cores on the two ported sections

Workflow speedup · AI agent (Claude Code)

months → weeks

- ~100k lines of Fortran navigated in days, not weeks
- instrumentation and analysis scripts written on demand
- solver study end-to-end: matrix dump → scipy → Fortran
- **≈ 4–5 engineer-months in 3 weeks** (team estimate)

Headline pair: jobs 52733 / 52710, ncpn31–34 · four part-time engineers, every patch reviewed and every number re-measured by a human

Workflow Speedup by AI Agent — Four Jobs It Took Off Our Hands

Humans chose what to try, judged the numerics and ran the jobs; the agent did the reading, the plumbing and the first draft

		by hand*	with agent
Navigate	~100k lines of Fortran: call graph, data flow, where each timer sits	3–4 weeks	**2 days**
Instrument	NVTX ranges, per-rank MPI trace, perf and timer parsers	2–3 weeks	**3 days**
Analyse	solver study: matrix dump → scipy preconditioners → time model → Fortran	4–6 weeks	**4 days**
Bookkeep	A/B job matrix, CHECK builds, result tables, release packaging	1–2 weeks	**1 day**

≈ 4–5 months of one engineer → 3 weeks. Not replaced: choosing what to try, judging round-off, the dead ends.

* team estimate of the same work by hand, to be confirmed in the report · NCHC's on-prem Muse-Spark-Open-816B-A42B is a candidate for code-navigation where source cannot leave the site

OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA

GPU Port — Transport and Backtracking on One B200 vs 544 Grace Cores

SCHISM has no GPU code, so this is a port: week 3 on a GB200 node (its CPU is Grace, so the Opt 1–8 build runs as is), the two biggest sections rewritten in CUDA C++ behind a Fortran bind(C) interface

0.19 s vs 0.178 s

Backtracking (timer 04) · per step

1 B200 already at the pace of 544 cores · one thread per trajectory, 72 M trajectories

0.32 s vs 0.164 s

Transport (timer 09) · per step

within 2× of 544 cores · one thread per (layer, edge) · kernels not yet tuned

0.51 s vs 0.34 s

both sections · per step

1 CPU core + 1 B200 against 4 Grace nodes running the Opt 1–8 build

ms per step (nsys)	4 Grace nodes · 544 cores	1 CPU core + 1 B200
Transport · compute (sub-step kernels)	114	219
Transport · MPI / CPU↔GPU transfer	56	31
Transport · other / CPU-side setup	26	105
Backtracking · compute	143	71
Backtracking · MPI wait / transfer	46	53
Both sections	**386**	**482**

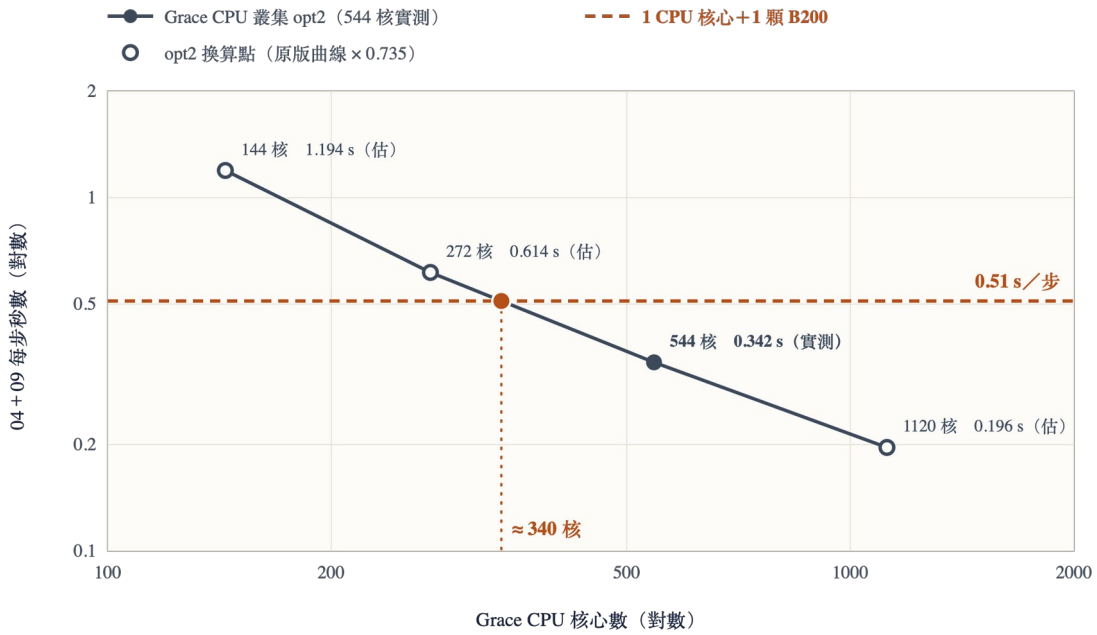
- **Compute is level:** 290 ms of kernels vs 258 ms on 544 cores
- **MPI became transfer:** 102 ms of allreduce/halo → 84 ms over C2C
- **Biggest gap:** 105 ms of CPU-side setup in Transport
- **Correctness:** FMA off → bit-identical; FMA on → inside noise

4 nodes = Opt 1–8 + NVTX job 50030 (rank 41, 180 steps) and job 52710 · GPU = jobs 689242 / 689392 (10-step median) · GB200 node, nvhpc 25.9 / CUDA 13.0

OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA

One B200 ≈ 340 Grace Cores — and What Is Still on the CPU

Backtracking + Transport per step placed on the CPU scaling curve of the optimized build · the rest of the step has not been ported yet



Backtracking + Transport per step vs Grace core count (CPU-optimized curve) · 1 B200 = 0.51 s

≈ 340 Grace cores

one B200 on the two ported sections · Backtracking ≈ 500, Transport ≈ 250

Per step (s)	4 Grace nodes · 544 cores	1 core + 1 B200
Transport + Backtracking	0.34	**0.51**
other sections	0.12	22.8 (1 CPU core)
Time step	0.46	23.3

- **Not ported:** MatrixPrep, Momentum, solver, levels — one CPU core, ~1000× slower than 544
- **Not tuned:** Transport flux kernel at ~10% of bandwidth

Two sections, one GPU, one week: the ported code already runs at the pace of ~340 Grace cores.

4 nodes = opt2 job 52710 (4 × 136, 180 steps) · GPU = job 689242 (10-step average) · CPU curve: Grace cluster nvtx scaling × opt2 ratio 0.735 · details in SCHISM 效能分析 ch. 14

OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA

Energy Efficiency

What we measured: node-hours saved on CPU, and how many Grace cores one B200 replaces on the ported sections

CPU tuning · same 4 Grace nodes

6.4 → 3.0 node-hours

per 8-day forecast (1.60 h → 0.74 h × 4 nodes) — 2.15× less energy for the same output

GPU port · Transport + Backtracking

1 B200 ≈ 2.4 Grace nodes

≈ 340 of 544 Grace cores (2 × 72 per node) for the same per-step work on the two ported sections

- **Not yet counted:** the rest of the time step still runs on one CPU core in the GPU build, so whole-step energy is not comparable yet
- **Measured, not estimated:** both numbers come from timer.out on the same case (jobs 52710, 689242)

Problems Encountered — and What Each One Taught Us

What cost us days during the three weeks, and the working habit it left behind

	Problem	Lesson
Legacy app	no GPU path in SCHISM; Fortran idioms compiled to memcpy	read the generated code, not the source — one switch per edit, CHECK build for each
Algorithm	CG on 540 ranks was sync-count bound, not compute bound	count syncs, not flops — 1,084 × 3 allreduce said “change the algorithm”
Tools	nvfortran -mp dropped vectorization inside parallel regions	measure, don't assume — hybrid was 20–25% slower, we stayed pure MPI
System	GB200 access only in week 3, 12 cores per GPU; node sets differ ~1%	every A/B in one job on one node set; port the two biggest sections first, verify with FMA off

Measured dead ends: compiler flags ≤ 1.1% · MPI+OpenMP hybrid 20–25% slower · cost-weighted partition −20% · questions raised with the mentors on Day 1

Final Thoughts

Three weeks, one production model, one method we keep

Delivered

- 2.15× on the production case, CWA accepted
- Transport + Backtracking on one B200 ≈ 340 Grace cores

Learned

- waiting first, compute second — placement was 40%
- profile every section, every week

Worth it?

- yes — the fast build is the operational build
- and the method outlives the hackathon

Thank you — NCHC, NVIDIA mentors, OpenACC.org, and the SCHISM community.

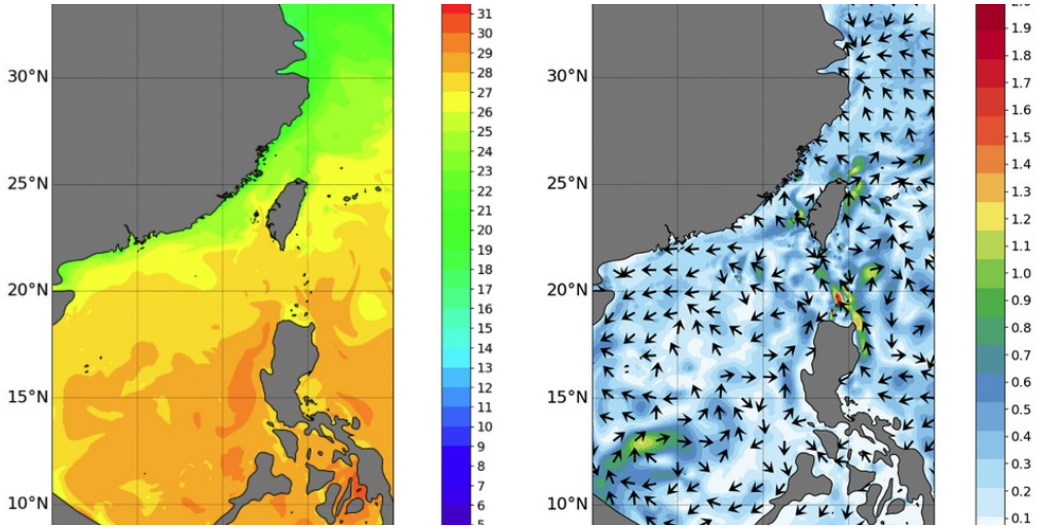
Team Baytech · Chihung Wang, Chiaying Tung, Haoyu Cheng, Yihsuan Kuan · mentor Shijie Wang (NVIDIA)

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Application Background — SCHISM, CWA Ocean Current Model

SCHISM: open-source 3D unstructured-grid ocean model. Baytech runs Taiwan's operational ocean-current forecast on it for the Central Weather Administration every day — 508k nodes, 48 levels, 8-day forecast in a fixed window.

Three motifs, three bottlenecks: water-level solver (MPI latency), backtracking (compute), tracer transport (memory bandwidth).



OCM daily analysis — sea-surface temperature and current speed, Western Pacific domain (CWA)

Hackathon Objectives and Approach

Make the production case faster on NVIDIA Grace CPU, then open a GPU path. MPI Fortran, nvfortran + HPC-X.

Profile in four layers; every change A/B on the same nodes behind its own compile switch; bit-identical first, solver changes judged against the model's own noise and the CWA 96 h acceptance.

AI agent (Claude Code): code navigation, instrumentation, the solver study end to end.

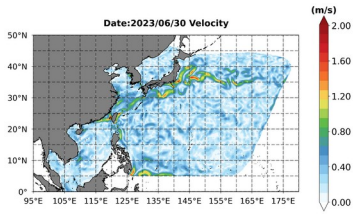
Technical Accomplishments and Impact

2.15× on the same 4 Grace nodes — 180 s → 84 s per 180 steps; 8-day forecast 1.60 h → 0.74 h; CWA accepted; two releases in operations.

GPU: one B200 ≈ 340 Grace cores on Transport + Backtracking, rewritten in CUDA C++ on a GB200 node; bit-identical with FMA off.

Workflow: ≈ 4–5 engineer-months of work in 3 weeks with the AI agent.

讓台灣每天的海流預報快 2.15 倍： SCHISM on NVIDIA Grace



研究領域：營運海洋數值預報（CWA 海流預報模式 OCM）

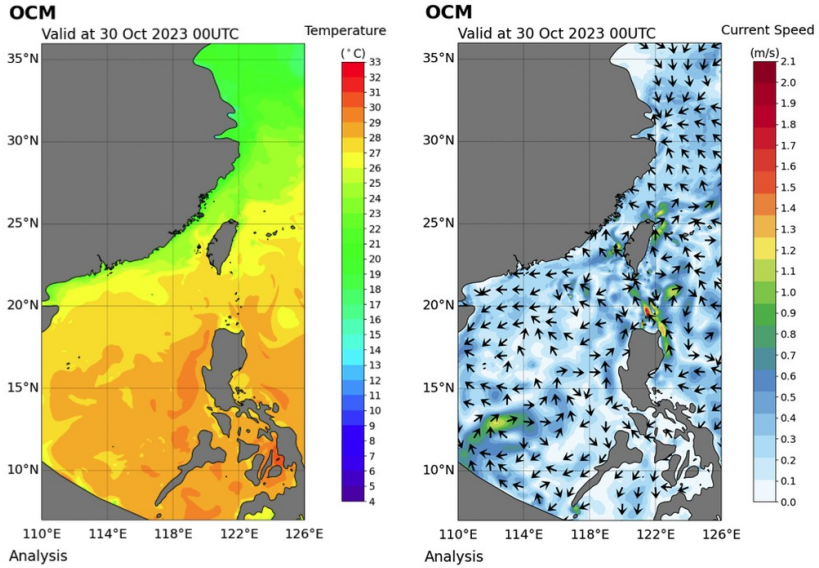
西太平洋計算域 95°E–180°E，50 萬網格點、 48 層，近岸 200 m 到外海 8 km 一張網格。

每天 5 天分析 +5 天預報，固定交付時窗。

Baytech 團隊來自西灣海環科技（國立中山大學海洋環境及工程學系衍生公司）， AI Agent 將 SCHISM 效能分析與最佳化的工作流程加速了約 5 倍； NVIDIA Grace CPU 將中央氣象署海流預報模式 SCHISM 的運算加速了 2.15 倍！！

台灣的颱風、油污、搜救與離岸風電都靠海流預報。中央氣象署的海流預報模式以開源模式 SCHISM 建置，由西灣海環每天營運：50 萬個網格點、 48 層，一次 8 天預報在 4 個節點上要跑 1.6 小時，而下一代系集同化要同時跑幾十個成員，算力就是瓶頸。這三週我們把模式搬到 NCHC 的 NVIDIA Grace 節點，四層剖析找出時間花在哪裡，八項最佳化讓同一組 4 個節點從 180 秒降到 84 秒，通過氣象署 96 小時驗收，程式已是營運版本。第三週再把最大的兩段改寫成 CUDA C++ 搬上 GB200： 一顆 B200 約等於 340 個 Grace 核心，關閉 FMA 時與 CPU 逐位元相同。 AI Agent 讀碼、插樁、帶著求解器研究從矩陣到新程式碼，讓四位兼職工程師在三週內完成估計要幾個月的工作。

補充數據 Grace 對 Xeon 8480+ 未調校快 2.03×（1 節點）～1.12×（10 節點）。節省來源：節點擺放 40%、氣象資料快取 28%、求解器 14%。正確性：程式修改逐位元相同（53 億次比對 0 差異）；CWA 96 小時水位 RMSE 1.18 mm （門檻 10 mm）。



CWA 海流預報模式（OCM）每日分析：海表溫度與流速

同一組 4 個 Grace 節點， 180 步	原版	最終版
Time stepping （秒）	180.1	83.6
加速倍數	1.00×	2.15×
8 天預報（小時）	1.60	0.74
Transport + Backtracking （秒／步）	544 核 0.34	1 顆 B200 0.51

Workload Grace CPU 同節點 2.15×； 一顆 B200 ≈ 340 個 Grace 核心

Workflow AI Agent： 估計 4–5 個月的單人工作在 3 週內完成


Thank You
OpenACC
More Science, Less Programming